

Handwritten Reflection Questions & Answers

1. Explain the difference between authentication and authorization.

Ans: Authentication is a process of verifying who a user actually is, usually by checking a username and password before allowing access to a system. Authorization, on the other hand, happens after authentication and decides what an already-verified user is allowed to do within the system. In my chat application, authentication happens when server checks the username and password sent by the client against the separate step, such as deciding whether a particular user is allowed to send private message or view certain chat history. My project mainly focuses on authentication, since every connected user currently has the same level of access once logged in, and there are no separate permission level defined for different users.

2. Why should passwords never be stored in plain text?

Ans: If passwords are stored in plain text, anyone who gains access to the storage file, such as `users.json` in my project, can immediately read every user's actual password without any extra effort. This is extremely dangerous because most people reuse the same password across multiple accounts, so a single leaked file could compromise a person's email, banking, or other account as well. Storing plain text passwords also meant that even the system administrator can see user passwords, which is a serious privacy and trust issue. In my implementation, I never store or print the actual password anywhere, not even in the security log file. Instead, I only

store and compare the SHA-256 hash of the password, so even if someone access users.json, they cannot directly see what the original password was.

3. What is the purpose of password hashing?

Ans: Password hashing converts a password into a fixed length string of characters using a one-way mathematical function, so that the original password can not be easily recovered from the hash. In my project, I use Python's `hashlib.sha256()` function to hash every password before comparing it with the stored value in users.json. When a user logs in, the server takes the password they typed, hashes it using the same SHA-256 function, and compares this new hash with the one already stored for that username. If both hashes matches exactly, the login is accepted, otherwise it is rejected. This way the actual password is never stored anywhere on the server, only its hash, which makes password hashing an essential technique for protecting user credentials even if the storage file itself is somehow exposed.

4. How does duplicate login prevention improve security?

Ans: Duplicate login prevention stops the same username from used to login from two different locations at the same time. In my server code, I maintain a set called `logged-in-users` that keep track of every username currently connected. Whenever a new login request comes in, the server checks if that username is already present in this set and if

it is, the new login attempt is rejected with a duplicate: login error. This is useful because if someone's credentials are somehow leaked or guessed, the real user would immediately notice something is wrong if they are suddenly logged out or cannot log in, since only one session per user is allowed. It also prevents confusion in the chat itself, where two people could otherwise pretend to be the same users at the same time.

5. Suggest two additional security improvements for your application.

Ans: The first improvement I would suggest is adding TLS encryption to the connection using Python's ssl module. Currently, even though passwords are hashed before storage, they are still sent from the client to the server as plain text over the network, which I confirmed myself while checking the login packets in Wireshark. Adding TLS would encrypt all traffic between the client and server, so even if someone captured the packets, they would not be able to read the login details. The second improvement would be adding salting to the password hashing process. Right now, two users with the same password would have the exact same SHA-256 hash, which is a small weakness. Adding a unique random salt for each user before hashing would make every hash different even for identical passwords, making the stored credentials much harder to attack.